

Lab 9: Data Visualization & Dashboards

Z5008 Big Data Lab · MTech Data Science & AI

Dr. Innocent Nyalala
`innocent@iitmz.ac.in`

School of Engineering & Science
IIT Madras Zanzibar

11 May 2026

Outline

Why Visualization Matters for Big Data

The Insight Gap

Raw numbers in a DataFrame tell you *facts*.
A well-crafted chart tells you a *story*.

East Africa e-commerce context:

- 100K+ orders/day across Kenya, Tanzania, Uganda, Rwanda
- Mobile money (M-Pesa, Airtel Money) dominates payments
- Seasonal spikes: Ramadan, Christmas, school-fees months
- Rural vs. urban purchasing patterns differ sharply

Lab 9 Toolkit

Layer	Tool
Static/pub	Matplotlib, Seaborn
Interactive	Plotly Express
Custom GO	Graph Objects
Dashboard	Dash
Geospatial	Folium, px.choropleth
BI/Ops	Superset, Grafana

Learning Goals

- Aggregate first, then plot
- Choose the right chart type
- Build interactive dashboards

The Aggregation-First Principle

Cardinal Rule

Aggregate in Spark. Plot with Pandas.

Never move millions of rows to the driver.

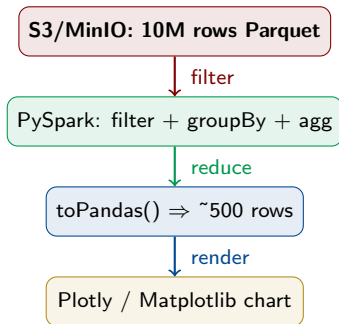
Safe pipeline:

- 1 Filter & select relevant columns in PySpark
- 2 `groupBy().agg()` to collapse to summary
- 3 `.limit(N)` as an absolute safety net
- 4 `.toPandas()` — result is now small
- 5 Plot with Plotly / Matplotlib

Memory budget:

1 row $\approx$ 200 bytes $\Rightarrow$

100K rows $\approx$ 20 MB (safe)



SparkSession Setup for Lab 9

```
1 from pyspark.sql import SparkSession
2 import pyspark.sql.functions as F
3
4 spark = SparkSession.builder \
5     .appName("Lab9-Visualization") \
6     .master("spark://spark-master:7077") \
7     .config("spark.hadoop.fs.s3a.endpoint", "http://minio:9000") \
8     .config("spark.hadoop.fs.s3a.access.key", "minioadmin") \
9     .config("spark.hadoop.fs.s3a.secret.key", "minioadmin") \
10    .config("spark.hadoop.fs.s3a.path.style.access", "true") \
11    .config("spark.hadoop.fs.s3a.impl",
12            "org.apache.hadoop.fs.s3a.S3AFileSystem") \
13    .config("spark.sql.shuffle.partitions", "8") \
14    .getOrCreate()
15
16 spark.sparkContext.setLogLevel("WARN")
17
18 # Load the 100K-row orders dataset from Lab 4
19 orders = spark.read.parquet("s3a://warehouse/lab04/orders_parquet/")
20 orders.printSchema()
21 print(f"Total rows: {orders.count():,}")
```

Safe toPandas() Patterns

```
1 # SAFE PATTERN 1      aggregate then convert
2 daily_rev = (
3     orders
4     .withColumn("date", F.to_date("order_timestamp"))
5     .groupBy("date", "country")
6     .agg(
7         F.sum("order_value_usd").alias("revenue"),
8         F.count("*").alias("num_orders")
9     )
10    .orderBy("date")
11 )
12 pdf_daily = daily_rev.toPandas()      # ~50 rows      fine
13 print(pdf_daily.shape)
14
15 # SAFE PATTERN 2      exploration sample (never production)
16 pdf_sample = orders.sample(fraction=0.01, seed=42).toPandas()
17
18 # DANGEROUS      do NOT do this with large tables
19 # pdf_bad = orders.toPandas()      # moves ALL rows to driver!
20
21 # SAFE PATTERN 3      categorical aggregation
22 pdf_cat = (
23     orders
24     .groupBy("country")
25     .agg(F.collect_set("product").alias("products"))
26     .toPandas()
```

Production-Quality Matplotlib Charts

```
1 import matplotlib.pyplot as plt
2 import matplotlib.ticker as mtick
3 import seaborn as sns
4
5 sns.set_theme(style="whitegrid", palette="muted", font_scale=1.1)
6 COLORS = {"KE": "#C5A059", "TZ": "#0a1628", "UG": "#00964F", "RW": "#E53935"}
7
8 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
9
10 # Left: grouped bar      revenue per country
11 pdf_ctype = pdf_daily.groupby("country")["revenue"].sum().reset_index()
12 axes[0].bar(pdf_ctype["country"],
13             pdf_ctype["revenue"] / 1e6,
14             color=[COLORS[c] for c in pdf_ctype["country"]],
15             edgecolor="white", linewidth=0.8)
16 axes[0].yaxis.set_major_formatter(
17     mtick.FuncFormatter(lambda x, _: f"${x:.1f}M"))
18 axes[0].set_title("Total Revenue by Country",
19                 fontsize=13, fontweight="bold")
20 axes[0].set_xlabel("Country"); axes[0].set_ylabel("Revenue (USD)")
21
22 # Right: line chart      daily orders
23 for country, grp in pdf_daily.groupby("country"):
```

Seaborn: Heatmap & KDE Distribution

```
1 import seaborn as sns, pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Pivot: country x payment_method order counts
5 pivot = pdf_cat.pivot_table(
6     values="num_orders", index="country",
7     columns="payment_method", aggfunc="sum", fill_value=0
8 )
9
10 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
11
12 # Heatmap annotated with actual counts
13 sns.heatmap(pivot, annot=True, fmt=",d", cmap="YlOrRd",
14             linewidths=0.5, ax=ax1,
15             cbar_kws={"label": "Order Count"})
16 ax1.set_title("Orders: Country x Payment Method", fontweight="bold")
17 ax1.set_xlabel("Payment Method"); ax1.set_ylabel("Country")
18
19 # KDE distribution of order values per country
20 for cty in ["KE", "TZ", "UG", "RW"]:
21     sub = pdf_sample[pdf_sample["country"] == cty]
22     sns.kdeplot(sub["order_value_usd"], ax=ax2,
23                 label=cty, fill=True, alpha=0.25, linewidth=2)
```

Plotly Express: Grouped & Stacked Bar Charts

```
1 import plotly.express as px
2
3 COLOR_MAP = {"KE": "#C5A059", "TZ": "#0a1628", "UG": "#00964F", "RW": "#E53935"}
4
5 # Grouped bar      revenue per category per country
6 fig = px.bar(
7     pdf_cat, x="category", y="revenue", color="country",
8     barmode="group",      # swap to "stack" for stacked
9     title="Revenue by Category and Country      East Africa E-commerce",
10    labels={"revenue": "Revenue (USD)", "category": "Product Category"},
11    color_discrete_map=COLOR_MAP,
12    text_auto=".2s",      # auto-annotate bars
13    height=500
14 )
15 fig.update_layout(
16     legend_title="Country",
17     plot_bgcolor="white",
18     xaxis=dict(gridcolor="#eeeeee"),
19     yaxis=dict(gridcolor="#eeeeee", tickprefix="$"),
20     font=dict(family="Inter, sans-serif", size=12)
21 )
22 fig.show()
```

Plotly Express: Line & Scatter

```
1 # Line chart with markers      daily revenue trend
2 fig_line = px.line(
3     pdf_daily, x="date", y="revenue", color="country",
4     markers=True, line_dash="country",
5     title="Daily Revenue Trend      East Africa E-commerce 2025",
6     labels={"revenue": "Revenue (USD)", "date": "Date"},
7     color_discrete_map=COLOR_MAP,
8     height=430
9 )
10 fig_line.update_traces(line_width=2.5)
11 fig_line.update_layout(plot_bgcolor="white")
12 fig_line.show()
13
14 # Scatter      order value vs delivery days
15 # colour = country, size = basket size, symbol = payment method
16 fig_scat = px.scatter(
17     pdf_sample, x="delivery_days", y="order_value_usd",
18     color="country", size="basket_size",
19     hover_name="order_id", symbol="payment_method",
20     color_discrete_map=COLOR_MAP,
21     trendline="ols",      # linear regression overlay per group
22     opacity=0.65,
23     title="Order Value vs Delivery Time (sampled)",
24     labels={"order_value_usd": "Order Value (USD)", "delivery_days": "Delivery Time (days)"},
25     color_discrete_map=COLOR_MAP,
26     height=430
27 )
28 fig_scat.show()
```

Plotly Express: Sunburst & Treemap

```
1 # Sunburst      country -> category -> payment_method
2 fig_sun = px.sunburst(
3     pdf_cat,
4     path=["country", "category", "payment_method"],
5     values="revenue",
6     color="revenue",
7     color_continuous_scale="YlOrRd",
8     title="Revenue Hierarchy: Country -> Category -> Payment",
9     height=560
10 )
11 fig_sun.update_traces(textinfo="label+percent entry")
12 fig_sun.show()
13
14 # Treemap      same hierarchy, different layout
15 fig_tree = px.treemap(
16     pdf_cat,
17     path=["country", "category"],
18     values="revenue",
19     color="num_orders",
20     color_continuous_scale="Viridis",
21     title="Treemap: Revenue (tile size) vs Orders (colour)",
22     height=500
```

Plotly Express: Box, Histogram, Choropleth

```
1 # Box plot      order value distribution with notch (CI on median)
2 fig_box = px.box(
3     pdf_sample, x="country", y="order_value_usd",
4     color="country", points="outliers", notched=True,
5     color_discrete_map=COLOR_MAP,
6     title="Order Value Distribution by Country",
7     labels={"order_value_usd": "Order Value (USD)"}
8 )
9 fig_box.show()
10
11 # Histogram      delivery days, overlaid by country
12 fig_hist = px.histogram(
13     pdf_sample, x="delivery_days", color="country",
14     nbins=30, barmode="overlay", opacity=0.60,
15     color_discrete_map=COLOR_MAP,
16     title="Delivery Days Distribution by Country"
17 )
18 fig_hist.show()
19
20 # Choropleth      African country-level revenue
21 fig_choro = px.choropleth(
22     pdf_country, locations="iso3", color="revenue",
23     locationmode="ISO-3", scope="africa",
```

Plotly Graph Objects: Manual Control

```
1 import plotly.graph_objects as go
2
3 fig = go.Figure()
4
5 for country in ["KE", "TZ", "UG", "RW"]:
6     mask = pdf_cat["country"] == country
7     fig.add_trace(go.Bar(
8         name=country,
9         x=pdf_cat[mask]["category"],
10        y=pdf_cat[mask]["revenue"],
11        text=pdf_cat[mask]["revenue"].map(lambda v: f"${v/1e3:.0f}K"),
12        textposition="inside",
13        marker_color=COLOR_MAP[country],
14        marker_line_width=0.5,
15        marker_line_color="white",
16    ))
17
18 fig.update_layout(
19     barmode="stack",
20     title=dict(text="Stacked Revenue by Category",
21               font=dict(size=18, family="Inter")),
22     legend=dict(orientation="h", yanchor="bottom",
23               y=1.02, xanchor="right", x=1),
```

Plotly Subplots: `make_subplots`

```

1 from plotly.subplots import make_subplots
2 import plotly.graph_objects as go
3
4 fig = make_subplots(
5     rows=2, cols=2,
6     subplot_titles=("Revenue Trend", "Orders by Country",
7                     "Payment Methods Share", "Category Heatmap"),
8     vertical_spacing=0.13, horizontal_spacing=0.09
9 )
10
11 # (1,1)      Line chart
12 fig.add_trace(go.Scatter(
13     x=pdf_daily["date"], y=pdf_daily["revenue"],
14     mode="lines+markers", name="Revenue", line_color="#C5A059"), row=1, col=1)
15
16 # (1,2)      Bar chart
17 fig.add_trace(go.Bar(
18     x=pdf_country["country"], y=pdf_country["num_orders"],
19     name="Orders", marker_color="#0a1628"), row=1, col=2)
20
21 # (2,1)      Pie
22 fig.add_trace(go.Pie(
23     labels=pdf_pay["method"], values=pdf_pay["count"],
24     title="Payment Methods Share",
25     sort=True,
26     pull=[0.05, 0.05, 0.05, 0.05],
27     pullwidth=[0.05, 0.05, 0.05, 0.05],
28     pullcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
29     pullwidthcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
30     pullwidthtextcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
31     pullwidthtextfont=["", "", "", ""],
32     pullwidthtextsize=[12, 12, 12, 12],
33     pullwidthtextweight=["normal", "normal", "normal", "normal"],
34     pullwidthtextalign=["center", "center", "center", "center"],
35     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
36     pullwidthtextdy=[-5, -5, -5, -5],
37     pullwidthtextdx=[0, 0, 0, 0],
38     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
39     pullwidthtextfontsize=[12, 12, 12, 12],
40     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
41     pullwidthtextalign=["center", "center", "center", "center"],
42     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
43     pullwidthtextdy=[-5, -5, -5, -5],
44     pullwidthtextdx=[0, 0, 0, 0],
45     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
46     pullwidthtextfontsize=[12, 12, 12, 12],
47     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
48     pullwidthtextalign=["center", "center", "center", "center"],
49     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
50     pullwidthtextdy=[-5, -5, -5, -5],
51     pullwidthtextdx=[0, 0, 0, 0],
52     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
53     pullwidthtextfontsize=[12, 12, 12, 12],
54     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
55     pullwidthtextalign=["center", "center", "center", "center"],
56     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
57     pullwidthtextdy=[-5, -5, -5, -5],
58     pullwidthtextdx=[0, 0, 0, 0],
59     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
60     pullwidthtextfontsize=[12, 12, 12, 12],
61     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
62     pullwidthtextalign=["center", "center", "center", "center"],
63     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
64     pullwidthtextdy=[-5, -5, -5, -5],
65     pullwidthtextdx=[0, 0, 0, 0],
66     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
67     pullwidthtextfontsize=[12, 12, 12, 12],
68     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
69     pullwidthtextalign=["center", "center", "center", "center"],
70     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
71     pullwidthtextdy=[-5, -5, -5, -5],
72     pullwidthtextdx=[0, 0, 0, 0],
73     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
74     pullwidthtextfontsize=[12, 12, 12, 12],
75     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
76     pullwidthtextalign=["center", "center", "center", "center"],
77     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
78     pullwidthtextdy=[-5, -5, -5, -5],
79     pullwidthtextdx=[0, 0, 0, 0],
80     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
81     pullwidthtextfontsize=[12, 12, 12, 12],
82     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
83     pullwidthtextalign=["center", "center", "center", "center"],
84     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
85     pullwidthtextdy=[-5, -5, -5, -5],
86     pullwidthtextdx=[0, 0, 0, 0],
87     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
88     pullwidthtextfontsize=[12, 12, 12, 12],
89     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
90     pullwidthtextalign=["center", "center", "center", "center"],
91     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
92     pullwidthtextdy=[-5, -5, -5, -5],
93     pullwidthtextdx=[0, 0, 0, 0],
94     pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
95     pullwidthtextfontsize=[12, 12, 12, 12],
96     pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
97     pullwidthtextalign=["center", "center", "center", "center"],
98     pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
99     pullwidthtextdy=[-5, -5, -5, -5],
100    pullwidthtextdx=[0, 0, 0, 0],
101    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
102    pullwidthtextfontsize=[12, 12, 12, 12],
103    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
104    pullwidthtextalign=["center", "center", "center", "center"],
105    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
106    pullwidthtextdy=[-5, -5, -5, -5],
107    pullwidthtextdx=[0, 0, 0, 0],
108    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
109    pullwidthtextfontsize=[12, 12, 12, 12],
110    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
111    pullwidthtextalign=["center", "center", "center", "center"],
112    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
113    pullwidthtextdy=[-5, -5, -5, -5],
114    pullwidthtextdx=[0, 0, 0, 0],
115    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
116    pullwidthtextfontsize=[12, 12, 12, 12],
117    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
118    pullwidthtextalign=["center", "center", "center", "center"],
119    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
120    pullwidthtextdy=[-5, -5, -5, -5],
121    pullwidthtextdx=[0, 0, 0, 0],
122    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
123    pullwidthtextfontsize=[12, 12, 12, 12],
124    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
125    pullwidthtextalign=["center", "center", "center", "center"],
126    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
127    pullwidthtextdy=[-5, -5, -5, -5],
128    pullwidthtextdx=[0, 0, 0, 0],
129    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
130    pullwidthtextfontsize=[12, 12, 12, 12],
131    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
132    pullwidthtextalign=["center", "center", "center", "center"],
133    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
134    pullwidthtextdy=[-5, -5, -5, -5],
135    pullwidthtextdx=[0, 0, 0, 0],
136    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
137    pullwidthtextfontsize=[12, 12, 12, 12],
138    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
139    pullwidthtextalign=["center", "center", "center", "center"],
140    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
141    pullwidthtextdy=[-5, -5, -5, -5],
142    pullwidthtextdx=[0, 0, 0, 0],
143    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
144    pullwidthtextfontsize=[12, 12, 12, 12],
145    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
146    pullwidthtextalign=["center", "center", "center", "center"],
147    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
148    pullwidthtextdy=[-5, -5, -5, -5],
149    pullwidthtextdx=[0, 0, 0, 0],
150    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
151    pullwidthtextfontsize=[12, 12, 12, 12],
152    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
153    pullwidthtextalign=["center", "center", "center", "center"],
154    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
155    pullwidthtextdy=[-5, -5, -5, -5],
156    pullwidthtextdx=[0, 0, 0, 0],
157    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
158    pullwidthtextfontsize=[12, 12, 12, 12],
159    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
160    pullwidthtextalign=["center", "center", "center", "center"],
161    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
162    pullwidthtextdy=[-5, -5, -5, -5],
163    pullwidthtextdx=[0, 0, 0, 0],
164    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
165    pullwidthtextfontsize=[12, 12, 12, 12],
166    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
167    pullwidthtextalign=["center", "center", "center", "center"],
168    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
169    pullwidthtextdy=[-5, -5, -5, -5],
170    pullwidthtextdx=[0, 0, 0, 0],
171    pullwidthtextfontcolor=["#0a1628", "#0a1628", "#0a1628", "#0a1628"],
172    pullwidthtextfontsize=[12, 12, 12, 12],
173    pullwidthtextfontweight=["normal", "normal", "normal", "normal"],
174    pullwidthtextalign=["center", "center", "center", "center"],
175    pullwidthtextbaseline=["bottom", "bottom", "bottom", "bottom"],
176    pullwidthtextdy=[-5, -5, -5, -5],
177    pullwidthtextdx=[0, 0, 0, 0],
178    pullwidthtextfontcolor=["#0a1628", "#0a1628
```

Plotly Animations: animation_frame

```
1 # Prepare monthly aggregation
2 pdf_monthly = (
3     orders
4     .withColumn("month", F.date_format("order_timestamp", "yyyy-MM"))
5     .groupBy("month", "country")
6     .agg(F.sum("order_value_usd").alias("revenue"),
7          F.count("*").alias("num_orders"),
8          F.avg("delivery_days").alias("delivery_days_avg"))
9     .orderBy("month")
10    .toPandas()
11 )
12
13 # Animated bar chart      revenue by country over months
14 fig_anim = px.bar(
15     pdf_monthly, x="country", y="revenue", color="country",
16     animation_frame="month",      # <- this drives the animation
17     animation_group="country",
18     range_y=[0, pdf_monthly["revenue"].max() * 1.15],
19     color_discrete_map=COLOR_MAP,
20     title="Monthly Revenue Animation --- East Africa 2025",
21     height=500
22 )
23 # Slow down playback
```

Plotly Animated Bubble Chart

```
1 # Animated bubble      delivery vs revenue, size = orders
2 fig_bubble = px.scatter(
3     pdf_monthly,
4     x="delivery_days_avg",
5     y="revenue",
6     animation_frame="month",
7     animation_group="country",
8     size="num_orders",
9     color="country",
10    hover_name="country",
11    size_max=60,
12    range_x=[0, pdf_monthly["delivery_days_avg"].max() * 1.2],
13    range_y=[0, pdf_monthly["revenue"].max() * 1.2],
14    color_discrete_map=COLOR_MAP,
15    title="Delivery Performance vs Revenue (Animated Bubble)",
16    height=500
17 )
18 fig_bubble.update_traces(marker=dict(opacity=0.75, line_width=1))
19 fig_bubble.show()
```

Design Tip

Dash: Architecture & Setup

Component Stack

dash — Flask-based Python server

dcc — Interactive widgets (Graph, Dropdown, Slider)

html — HTML layout building blocks

dash.dependencies — Input, Output, State

Request cycle:

- 1 User moves widget
- 2 Browser POSTs to Flask
- 3 Python callback runs
- 4 Returns serialised Plotly JSON
- 5 Browser re-renders chart

```
# Install in JupyterLab container
pip install dash dash-bootstrap-components

# docker-compose.yml      add port mapping
services:
  jupyterlab:
    ports:
      - "8888:8888"
      - "8050:8050"      # Dash app

# Run standalone app
python app.py
# http://localhost:8050

# Run inline in notebook
app.run(debug=True, port=8050,
        jupyter_mode="tab")
```

Port 8050

Dash default port. Already mapped in the Lab

Dash: Minimal App Layout

```
1 from dash import Dash, dcc, html, Input, Output, callback
2 import plotly.express as px
3 import pandas as pd
4
5 app = Dash(__name__)
6
7 COUNTRIES = ["All", "KE", "TZ", "UG", "RW"]
8
9 app.layout = html.Div([
10     html.H1("East Africa E-commerce Dashboard",
11             style={"textAlign": "center", "color": "#800000",
12                   "fontFamily": "Inter, sans-serif"}),
13
14     html.Div([
15         html.Label("Country:", style={"fontWeight": "bold"}),
16         dcc.Dropdown(
17             id="country-dd",
18             options=[{"label": c, "value": c} for c in COUNTRIES],
19             value="All", clearable=False,
20             style={"width": "200px"}
21         ),
22     ], style={"display": "flex", "alignItems": "center", "gap": "10px",
23             "padding": "10px 40px"}),
```

Dash: Callbacks — Input/Output Pattern

```
1 @callback(  
2     Output("revenue-chart", "figure"),  
3     Output("orders-chart", "figure"),  
4     Input("country-dd", "value")  
5 )  
6 def update_charts(selected_country):  
7     if selected_country == "All":  
8         filtered = pdf_daily  
9     else:  
10         filtered = pdf_daily[pdf_daily["country"] == selected_country]  
11  
12     fig_rev = px.line(  
13         filtered, x="date", y="revenue", color="country",  
14         title=f"Daily Revenue --- {selected_country}",  
15         markers=True, color_discrete_map=COLOR_MAP,  
16         labels={"revenue": "Revenue (USD)", "date": "Date"}  
17     )  
18     fig_rev.update_layout(plot_bgcolor="white")  
19  
20     fig_ord = px.bar(  
21         filtered.groupby("country")["num_orders"].sum().reset_index(),  
22         x="country", y="num_orders", color="country",  
23         title=f"Total Orders --- {selected_country}",
```

Dash: Advanced — RangeSlider, DatePicker, DataTable

```
1 from dash import dash_table
2 import dash_bootstrap_components as dbc
3
4 app2 = Dash(__name__, external_stylesheets=[dbc.themes.BOOTSTRAP])
5 CATS = sorted(pdf_cat["category"].unique().tolist())
6
7 app2.layout = dbc.Container([
8     dbc.Row([
9         dbc.Col([html.Label("Category:"),
10             dcc.Dropdown(id="cat-dd", options=CATS,
11                 multi=True, placeholder="All categories")], width=4),
12         dbc.Col([html.Label("Order Value Range (USD):"),
13             dcc.RangeSlider(id="val-slider",
14                 min=0, max=1000, step=25, value=[0,1000],
15                 marks={0: "$0", 250: "$250", 500: "$500", 1000: "$1K"})], width=5),
16         dbc.Col([html.Label("Date Range:"),
17             dcc.DatePickerRange(id="date-range",
18                 start_date="2025-01-01",
19                 end_date="2025-12-31")], width=3),
20     ], className="mb-3"),
21     dbc.Row([
22         dbc.Col(dcc.Graph(id="main-chart"), width=8),
23         dbc.Col(dcc.Graph(id="pie-chart"), width=4),
24     ]),
25 ])
```

Folium: Bubble Map of East Africa

```
1 import folium
2 from folium.plugins import MarkerCluster
3
4 # City coordinates, country, and order counts
5 cities = {
6     "Nairobi": {"lat": -1.286, "lon": 36.817, "cty": "KE", "orders": 45000},
7     "Mombasa": {"lat": -4.043, "lon": 39.668, "cty": "KE", "orders": 18000},
8     "Dar es Salaam": {"lat": -6.792, "lon": 39.208, "cty": "TZ", "orders": 38000},
9     "Zanzibar City": {"lat": -6.165, "lon": 39.202, "cty": "TZ", "orders": 8000},
10    "Kampala": {"lat": 0.347, "lon": 32.583, "cty": "UG", "orders": 22000},
11    "Kigali": {"lat": -1.943, "lon": 30.059, "cty": "RW", "orders": 15000},
12 }
13 COLORS = {"KE": "#C5A059", "TZ": "#0a1628", "UG": "#00964F", "RW": "#E53935"}
14
15 m = folium.Map(location=[-2.5, 35.0], zoom_start=5,
16               tiles="CartoDB positron")
17
18 for name, info in cities.items():
19     radius = (info["orders"] / 1000) ** 0.55 * 5
20     folium.CircleMarker(
21         location=[info["lat"], info["lon"]], radius=radius,
22         color=COLORS[info["cty"]], fill=True, fill_opacity=0.75,
23         popup=folium.Popup()
```

Plotly Choropleth: African Country Data

```
1 import plotly.express as px, pandas as pd
2
3 pdf_africa = pd.DataFrame({
4     "iso3":      ["KEN", "TZA", "UGA", "RWA", "ETH", "GHA", "NGA", "ZAF"],
5     "country":   ["Kenya", "Tanzania", "Uganda", "Rwanda",
6                  "Ethiopia", "Ghana", "Nigeria", "South Africa"],
7     "revenue":   [2.1e6, 1.8e6, 0.9e6, 0.5e6, 0.3e6, 0.7e6, 1.2e6, 0.4e6],
8     "num_orders": [120000, 95000, 48000, 28000, 15000, 38000, 62000, 18000],
9     "mobile_pct": [78, 72, 65, 81, 45, 60, 55, 40], # % mobile-money
10 })
11
12 fig = px.choropleth(
13     pdf_africa,
14     locations="iso3", locationmode="ISO-3",
15     color="revenue", scope="africa",
16     hover_name="country",
17     hover_data={"num_orders":":", "mobile_pct":":.0f%", "revenue":":$, .0f"},
18     color_continuous_scale="YlOrRd",
19     title="E-commerce Revenue Across Africa --- 2025",
20     labels={"revenue": "Revenue (USD)"},
21     height=530
22 )
23 fig.update_geos(showcoastlines=True, coastlinecolor="white",
```

Apache Superset: Self-Serve BI

What is Apache Superset?

- Open-source BI & data exploration platform
- Connects to Spark Thrift Server, PostgreSQL, Druid, BigQuery, ...
- Drag-and-drop chart builder (no code for analysts)
- Built-in SQL IDE (SQL Lab)
- Role-based access control
- Embeddable iframes for portal integration

Add to docker-compose.yml:

```
superset:  
  image: apache/superset:3.1.0  
  ports: ["8088:8088"]  
  environment:  
    - SUPERSET_SECRET_KEY=lab9supersecret
```

Superset Workflow

- 1 Settings → **Database Connections** → Add
- 2 Choose driver: Spark SQL, PostgreSQL, etc.
- 3 Create a **Dataset** from a table or SQL query
- 4 **Charts** → pick chart type, drag metrics
- 5 Build a **Dashboard** by arranging charts
- 6 Set refresh interval (e.g. every 30 s)

Grafana + Prometheus: Operational Dashboards

Use Cases in Big Data:

- Monitor Spark job stages, GC pauses, task failures
- Track cluster CPU/RAM/disk utilisation
- Alert when job exceeds SLA duration
- Real-time streaming pipeline throughput
- Kafka consumer lag monitoring

Sample PromQL queries:

```
# JVM heap used on executor nodes
jvm_memory_used_bytes{area="heap"}
```

```
# CPU usage rate (5-min window)
avg(rate(node_cpu_seconds_total
{mode!="idle"}[5m])) * 100
```

Grafana Panel Types

Panel	Best for
Time series	Metrics over time
Stat	Single KPI value
Gauge	% utilisation
Bar chart	Category compare
Table	Tabular drill-down
Heatmap	Density over time
Logs	Log stream
Alerting	Threshold breach

Enable Spark Metrics

```
.config("spark.metrics.conf",
```

Big Data Visualization Principles

Aggregation Strategies

- **GROUP BY**: exact sums, counts, averages
- **Sampling**: `.sample(fraction=0.01)` for EDA
- **Binning**: histograms on bucketed values
- **Approximate**:
`approx_count_distinct()`,
HyperLogLog sketches
- **Window avg**: rolling means smooth noisy series

Overplotting Solutions

- Alpha transparency (`opacity=0.1`)

Colorblind-Safe Palette Rules

- Avoid red–green combos (affects ~8% of men)
- Prefer **Okabe–Ito** or **Viridis**
- **Sequential** (magnitude): YlOrRd, Blues, Viridis
- **Diverging** (neg/pos): RdBu, PiYG, coolwarm
- **Categorical**: qualitative palettes, max 7–8 hues
- Test at coblis simulator

Data Ink Ratio (Tufte)

Chart Selection Guide

Question → Chart type

- **Comparison:** Bar chart, Radar
- **Trend over time:** Line, Area
- **Correlation:** Scatter, Bubble
- **Part-of-whole:** Pie / Donut (≤ 5 slices), Treemap
- **Distribution:** Histogram, KDE, Box, Violin
- **Geographic:** Choropleth, Bubble map
- **Matrix:** Heatmap, Parallel coords
- **Flow/hierarchy:** Sankey, Treemap,

When NOT to Use

Pie	>5 slices; not part-of-whole
3-D bar	Always misleading
Dual axis	Misleads scale comparison
Raw scatter	>10K points (overplot)
Line chart	Unordered categorical x-axis

Actionable vs. Decorative

Actionable dashboard:

- Answers a specific decision question
- Refreshes with live data

Lab 9 Tasks Summary

Today's Lab (2 hours 40 minutes)

- 1 **Pipeline** — Load orders parquet → `groupBy` → `toPandas()`
- 2 **Matplotlib/Seaborn** — 3-panel subplot: revenue bar, orders line, heatmap
- 3 **Plotly Express** — bar (grouped & stacked), line, scatter, sunburst, choropleth
- 4 **Subplots** — 2×2 executive dashboard with `make_subplots`
- 5 **Animation** — Monthly revenue animated bar chart with `animation_frame`
- 6 **Dash App** — Dropdown + RangeSlider + 2 synchronised callback charts
- 7 **Folium** — East Africa bubble map by city, saved as `.html`

Assignment (due Sun 17 May 23:59 EAT)

100 marks | 5 tasks | submit notebook +
HTML exports to:
`s3a://submissions/week09/<rollno>/`

Grading Criteria

Correctness · Chart quality ·
Interactivity depth · Code clarity ·
Storytelling & reflection quality

Docker Service Reference

Active Services — Lab 9

Service	URL	Credentials	Purpose
JupyterLab	localhost:8888	token in logs	Notebooks
Spark Master	localhost:8080	—	Cluster UI
Spark RPC	spark-master:7077	—	Submit jobs
Spark UI	localhost:4040	—	Job monitoring
MinIO Console	localhost:9001	minioadmin/minioadmin	Object storage
Kafka Broker	kafka:9092	—	Streaming bus
Grafana	localhost:3000	admin/admin	Dashboards
Prometheus	localhost:9090	—	Metrics store

Data Locations

Orders dataset:

`s3://warehouse/lab04/orders_parquet/`

Note

URLs may differ on the exam machines. Use

`docker compose ps` and container logs to

Practical Exam (2 hours): End-to-End Big Data Pipeline

Goal

Build a **working mini data platform** from ingestion to dashboards: **Kafka** → **Spark** → **MinIO/Postgres** → **Prometheus/Grafana**.

What you are allowed to use

- Internet + any AI tools (must cite prompts in README)
- Any libraries already available in the lab Docker images

What you must demonstrate

- Services running (compose), data flowing, storage visible
- Streaming + batch/triggered job + monitoring dashboard

Deliverables (submit a ZIP)

- Notebook or scripts + an `exam_README.md`
- Your Kafka topic name(s) + schema
- Evidence: screenshots (MinIO, Grafana, Spark UI)
- Exported Grafana dashboard JSON

Exam Data: “Clickstream + Orders” (Provided Generator)

Input Stream

You will produce JSON events to Kafka topic `exam.clickstream.<rollno>`.

Event fields (minimum):

- `event_time` (ISO timestamp), `user_id`, `country`, `event_type`
- `product_id`, `category`, `price_usd`

Rule

Your pipeline must handle **bad records** (missing fields / wrong types): count them and store them to a quarantine/ location.

Task A (25 marks): Kafka Ingestion + Streaming Storage

Build and prove:

- 1 A producer generates $\geq 2,000$ events in 2–3 minutes.
- 2 Spark Structured Streaming reads Kafka.
- 3 Write raw events to MinIO as Parquet, partitioned by `dt=YYYY-MM-DD`.
- 4 Write quarantine (bad records) to `s3a://warehouse/exam/quarantine/`.

Evidence

Screenshot: Kafka topic has messages; MinIO shows new Parquet files.

Task B (25 marks): Real-Time Aggregation + Serving Store

Compute (streaming aggregation):

- KPI1: events/sec (throughput)
- KPI2: revenue/minute by country (sum of price_usd for purchase)
- KPI3: top-5 categories in last 10 minutes

Serve to PostgreSQL:

- Upsert a table `exam_kpis_<rollno>` every micro-batch

Rule

Your serving table must be queryable from SQL (show at least 2 queries).

Task C (20 marks): Triggered Batch Job + Data Quality Checks

Create a triggered job (choose one):

- 1 **Airflow DAG** that runs once and materialises a daily summary table, or
- 2 A **scheduled Spark job** (cron-style) that writes a batch summary.

Minimum data-quality checks:

- Null-check rate (country, event_time)
- Outlier check: price_usd > 10000 (count)
- Duplicate check on (user_id, event_time, product_id)

Evidence

Screenshot: DAG run success (or job log) + output table/files exist.

Task D (20 marks): Prometheus Metrics + Grafana Dashboard

Expose at least 3 custom metrics (your choice), e.g.:

- `events_processed_total`
- `bad_records_total`
- `batch_duration_seconds`

Build a Grafana dashboard with 4 panels:

- 1 Throughput (time series)
- 2 Bad records (stat)
- 3 Revenue/min by country (time series)
- 4 Top categories (table)

Alert

Create one alert rule: bad records $> 1\%$ for 2 minutes.

Submission Checklist (ZIP) + Marking Rubric

ZIP must contain:

- exam_README.md (setup, commands, AI prompt citations)
- producer.py (or notebook cell) + topic name
- streaming_job.py (or notebook) + checkpoint path
- dq_job.py or dags/exam_dag.py
- grafana_dashboard.json
- screenshots/ (MinIO + Grafana + Spark UI)

Rubric

- Task A: 25
- Task B: 25
- Task C: 20
- Task D: 20
- Clarity (README + reproducibility): 10

Good luck!

Good luck!

Submit a reproducible ZIP (it should run on a clean machine).

Dr. Innocent Nyalala — `innocent@iitmz.ac.in`